Ryan Mahmoudi
Prelim ArduPilot Research

Pymavlink Python guide

- Use 64-bit Raspberry Pi OS and a Python virtual environment.
- Install pymavlink; pin the tested library, Python, and ArduCopter versions.
- Import mavutil; create a mavlink_connection() for UDP or serial.
- Wait for a heartbeat with a deadline; verify autopilot and system identity.
- Use discovered target IDs and ArduPilot dialect definitions.
- Proposed stack: direct pymavlink control with Mission Planner monitoring.

ArduPilot SITL

- Run SITL on a development computer after installing its build environment.
- Launch: sim_vehicle.py -v ArduCopter -f quad --console --map.
- Forward MAVProxy output to the Pi IP and a chosen UDP port.
- Use separate outputs for the application and ground station.
- The stock quad validates sequencing; custom performance needs a matched model.

Raspberry Pi UART configuration

- Choose USB serial or a configured 3.3 V UART connection.
- Cross TX/RX, share ground, and verify flight-controller connector pinouts.
- Pi 5 primary UART defaults to its debug header; select the intended interface.
- Disable serial console use; check device permissions and stable device paths.
- Store transport, device path, and baud in configuration.

ArduPilot Raspberry Pi connection

- Configure the connected port for MAVLink 2; SERIALx_PROTOCOL = 2.
- Match SERIALx_BAUD to the Pi; verify the board's logical port mapping.
- Older Pi device-path and power examples require Pi 5-specific checking.
- Power the Pi from an appropriate regulator, not the TELEM supply.
- Pi sends high-level commands; ArduCopter controls ESC outputs.

Copter Guided commands

- States: connect → ready → GUIDED → arm → take off → hover → LAND → disarmed.
- Arm with MAV_CMD_COMPONENT_ARM_DISARM; verify the armed heartbeat flag.
- Send MAV_CMD_NAV_TAKEOFF with target altitude above home in param7.
- Confirm climb from telemetry; command acceptance does not prove motion.
- Replace fixed sleeps with measured conditions and deadlines.
- Log transitions, command results, altitude, drift, attitude, battery, and mode.

Prearm checks

- Keep flight arming checks enabled; resolve reported failures.
- Capture STATUSTEXT and verify calibration, battery, and estimator readiness.
- Require valid horizontal positioning for the planned Guided test.
- Abort before arming if readiness or fresh telemetry is missing.

Copter Land mode

- Send MAV_CMD_NAV_LAND or request LAND mode; verify the actual mode.
- Check landing-speed parameters and units for the installed firmware.
- Watch landed state, vertical motion, and armed status.
- Require actual touchdown and disarming before reporting success.
- Taut restraints can distort landing detection; allow real gear contact.

Optical flow setup

- Indoor option: optical flow plus range sensing and configured EKF sources.

Ryan Mahmoudi
Prelim ArduPilot Research

- A rangefinder alone cannot establish horizontal position.
- Validate EKF origin and local-position quality before Guided flight.
- Use consistent coordinates; LOCAL_POSITION_NED uses positive down.
- Measure climb relative to start; detect estimator resets or jumps.

Requesting telemetry

- Use MAV_CMD_SET_MESSAGE_INTERVAL to request individual message rates.
- Request HEARTBEAT, LOCAL_POSITION_NED, ATTITUDE, and BATTERY_STATUS as needed.
- Request EXTENDED_SYS_STATE for landed state where supported.
- Check actual arrival rates; ground-station stream requests can interfere.
- Proposed: one receive loop dispatches messages to telemetry and ACK handlers.

MAVLink command protocol

- Match COMMAND_ACK to the outstanding command and sending vehicle.
- Handle accepted, rejected, in-progress, and timeout results explicitly.
- Verify actual mode and armed state independently of command responses.
- Use bounded retries; avoid duplicate takeoff requests after ambiguous results.

Python concurrency and timeouts

- Run sequencing, telemetry, and watchdogs concurrently.
- Keep blocking pymavlink reads off the asyncio loop; use a worker and queue.
- Use monotonic deadlines and per-message freshness checks.
- Proposed modules: connection, telemetry, sequence, watchdog, logger.
- Cancelling Python does not stop the aircraft; implement tested recovery.

Copter GCS failsafe

- Configure FS_GCS_ENABLE, FS_GCS_TIMEOUT, and relevant failsafe options.
- Send periodic application heartbeats; verify which sender the firmware monitors.
- Confirm a surviving GCS or router cannot mask a dead control application.
- Test Pi termination, link loss, and stale data in SITL before hardware.
- Choose a fixture-compatible response; keep independent RC takeover available.

ArduPilot flight logs

- Save onboard DataFlash logs, MAVLink telemetry, and application CSV or JSON.
- Archive parameters, firmware version, code commit, mass, and battery state.
- Compare target/measured height, XY drift, attitude, and motor outputs.
- Proposed pass: sequence completes within limits, without faults, then disarms.
- Advance: SITL → prop-free integration → supervised restrained flight.
- Require three consecutive passing runs before expanding test limits.

Guided setpoints

- Optional later step: SET_POSITION_TARGET_LOCAL_NED with explicit type masks.
- Provide all three axes for each enabled position, velocity, or acceleration group.
- Resend velocity/acceleration commands at least once per second.
- Check GUID_TIMEOUT; Guided_NoGPS provides attitude control, not XY holding.